

PIZZA SALES REPORT FOR THE YEAR 2015

By: Magaji Bashiru

VEPH/20C/DA119

Database & Visualization Tools

Practical Project

Project Overview

An independent project designed to develop practical competency in integrating relational databases with Power BI visualization tools (an essential skill in modern data analytics and business intelligence roles). The project replicates a real-world case study sourced from a YouTube tutorial, where a pizza sales dataset is queried using SQL and subsequently visualized in Power BI to generate business insights. This approach provides hands-on experience in working with real-world data and applying end-to-end data analysis techniques.

Furthermore, the exercise reflects typical workplace scenarios in which data analysts are required to extract, transform, and communicate data insights effectively to both technical and non-technical stakeholders. The primary objective of this project is to analyze pizza sales data to generate actionable insights that support business decision-making and performance evaluation. The specific objectives are outlined in the problem statement below.

Problem Statement

KPI Requirement

We need to analyze key indicators for our pizza sales to gain insights into our business performance. Specifically, we want to calculate the following metrics:

1. **Total Revenue:** the sum of the total price of all pizza orders.
2. **Average order value:** the average amount spent per order, calculated by dividing the total revenue by the total number of orders.
3. **Total pizza sold:** the sum of the quantity of all pizzas sold.
4. **Total orders:** the total number of orders placed.
5. **Average pizza per order:** the average number of pizzas sold per order, calculated by dividing the total number of pizzas sold by the total number of orders.

Chart Requirements

We would also like to visualize various aspect of our pizzas sale to gain insights and understand key trends. We have identified the following requirement for creating charts.

1. Daily Trend for Total Orders

Create a column-chart that display the daily trend of total order over a specific time period. This chart will help us identify any patterns or fluctuations in order volume on daily basis.

2. Monthly Trend for Total Orders

Create a line chart that illustrate the monthly trend of total order throughout the year. This chart will allow us identify peak seasons or period of high order activities.

3. Percentage of Sales by Pizza Category

Create a doughnut-chart that shows the distribution of sales across different pizza category. This chart will provide insights into the popularity of various pizza categories and their contribution to overall sales.

4. Percentage of Sales by Pizza Size

Generate a pie-chart that represent the percentage of sales attributed to different pizza sizes. This chart will help us understand customer preferences for pizza size and their impact on sales.

5. Total Pizzas Sold by Pizza Category

Create a funnel chart that represent the total numbers of pizzas sold for each pizza category. This chart will allow us to compare the sales performance of different pizza categories.

6. Top Five Best Sellers by Revenue, Total Quantity and Total Order

Create a bar-chart highlighting the top 5 best-selling pizzas based on the revenue, Total quantity, Total orders. This chart will help us identify the most popular pizza options.

7. Bottom five worst sellers by Revenue, Total quantity and Total order

Create a bar-chart highlighting the top 5 worst-selling pizzas based on the revenue, Total quantity, Total orders. This chart will enable us to identify the underperforming or less popular pizza options.

Data Source

The dataset is a time-series dataset covering the period from January to December 2015, containing daily pizza sales records in CSV format. It is structured in rows and columns, comprising over 48,000 records and 12 attributes. The dataset was sourced from Kaggle.com.

Data Preparation

The dataset was cleaned and validated on Excel prior to database import. A check for blank cells using **Find and Replace (Special → Blanks)** returned no empty entries, and the **Remove Duplicates** tool confirmed that no duplicate records existed.

Data standardization was performed on the pizza name column by removing the prefix “The” and applying the **TRIM** function to eliminate excess spaces. In the pizza ingredients column, a typographical error (“?dura”) was corrected using **Find and Replace All**. Following these preprocessing steps, an initial attempt made to import the dataset (over 48,000 rows) into PostgreSQL failed. Upon further inspection, inconsistent date formats were identified as the cause. The issue was resolved by standardizing the date column using the **Text to Columns** feature under the data tab.

After ensuring uniformity in the date format, the dataset was reloaded into PostgreSQL and the process was completed successfully.

Additional data transformation was carried out in Power BI using Power Query. This included the creation of new columns, as well as the development of calculated columns and measures using **DAX functions** to derive key performance indicators (KPIs) for analysis.

Database Design

The table named `pizza_sales` has 48620 rows and 12 columns which contains three **INT**, two **VARCHAR**, two **FLOAT**, three **TEXT**, a **DATE** and **TIME**.

	<code>pizza_id</code> integer	<code>order_id</code> integer	<code>pizza_name_id</code> character varying	<code>quantity</code> integer	<code>order_date</code> date	<code>order_time</code> time without time zone	<code>unit_price</code> double precision	<code>total_price</code> double precision	<code>pizza_size</code> character varying	<code>pizza_category</code> text	<code>pizza_ingredients</code> text
1	1	1	hawaiian_m	1	2015-01-01	11:38:36	13.25	13.25	M	Classic	Sliced Ham
2	2	2	classic_dlx_m	1	2015-01-01	11:57:40	16	16	M	Classic	Pepperoni
3	3	2	five_cheese_l	1	2015-01-01	11:57:40	18.5	18.5	L	Veggie	Mozzarella
4	4	2	ital_supr_l	1	2015-01-01	11:57:40	20.75	20.75	L	Supreme	Calabrese
5	5	2	mexicana_m	1	2015-01-01	11:57:40	16	16	M	Veggie	Tomatoes

SQL Logic

Creating Table – This allows to import dataset into the platform for further query.

```
CREATE TABLE pizza_sales (  
    pizza_id INT,  
    order_id INT,  
    pizza_name_id VARCHAR,  
    quantity INT,  
    order_date DATE,  
    order_time TIME WITHOUT TIME ZONE,  
    unit_price FLOAT,  
    total_price FLOAT,  
    pizza_size VARCHAR,  
    pizza_category TEXT,
```

```

pizza_ingredients TEXT,
pizza_name TEXT
);

```

	pizza_id integer	order_id integer	pizza_name_id character varying	quantity integer	order_date date	order_time time without time zone	unit_price double precision	total_price double precision	pizza_size character varying	pizza_category text	pizza_ingre text
1	1	1	hawaiian_m	1	2015-01-01	11:38:36	13.25	13.25	M	Classic	Sliced Han
2	2	2	classic_dlx_m	1	2015-01-01	11:57:40	16	16	M	Classic	Pepperoni,
3	3	2	five_cheese_l	1	2015-01-01	11:57:40	18.5	18.5	L	Veggie	Mozzarella
4	4	2	ital_supr_l	1	2015-01-01	11:57:40	20.75	20.75	L	Supreme	Calabrese
5	5	2	mexicana_m	1	2015-01-01	11:57:40	16	16	M	Veggie	Tomatoes,

A. KPI QUERIES

1. Total Revenue:

```

SELECT SUM(total_price) AS Total_Revenue
FROM pizza_sales;

```

	total_revenue double precision
1	817860.0499999928

2. Average Order Value

```

SELECT (SUM(total_price) / COUNT(DISTINCT order_id)) AS Average_order_Value
FROM pizza_sales

```

	average_order_value double precision
1	38.307262295081635

3. Total Pizzas Sold

```

SELECT SUM(quantity) AS Total_pizza_sold
FROM pizza_sales

```

	total_pizzas_sold bigint
1	49574

4. Total Orders

```
SELECT COUNT(DISTINCT order_id) AS Total_Order  
FROM pizza_sales
```

	total_order bigint
1	21350

5. Average Pizzas Per Order

```
SELECT CAST(CAST(SUM(quantity) AS DECIMAL(10,2)) /  
CAST(COUNT(DISTINCT order_id) AS DECIMAL(10,2)) AS DECIMAL(10,2))  
AS Average_Pizzas_Per_Order  
FROM pizza_sales
```

	average_pizzas_per_order numeric (10,2)
1	2.32

B. Chart Queries

1. Daily Trend for Total Orders

```
SELECT  
    TO_CHAR(order_date, 'Day') AS order_day,  
    COUNT(DISTINCT order_id) AS total_orders  
FROM  
    pizza_sales  
GROUP BY  
    TO_CHAR(order_date, 'Day'),  
    EXTRACT(DOW FROM order_date);
```

Output:

	order_day text	total_orders bigint
1	Friday	3538
2	Monday	2794
3	Saturday	3158
4	Sunday	2624
5	Thursday	3239
6	Tuesday	2973
7	Wednesday	3024

2. Monthly Trend for Orders

SELECT

TO_CHAR(order_date, 'Day') AS order_day,
COUNT(DISTINCT order_id) AS total_orders

FROM

pizza_sales

GROUP BY

TO_CHAR(order_date, 'Day');

Output

	order_month text	total_orders bigint
1	April	1799
2	August	1841
3	December	1680
4	February	1685
5	January	1845
6	July	1935
7	June	1773
8	March	1840
9	May	1853
10	November	1792
11	October	1646
12	September	1661

3. Percentage of Sales by Pizza Category

```
SELECT pizza_category, CAST(SUM(total_price) AS DECIMAL(10,2)) as total_revenue,  
CAST(SUM(total_price) * 100 / (SELECT SUM(total_price) from pizza_sales) AS  
DECIMAL(10,2)) AS Percentage  
FROM pizza_sales  
GROUP BY pizza_category
```

Output

	pizza_category text	total_revenue numeric (10,2)	percentage numeric (10,2)
1	Supreme	208197.00	25.46
2	Chicken	195919.50	23.96
3	Veggie	193690.45	23.68
4	Classic	220053.10	26.91

4. Percentage of Sales by Pizza Size

```
SELECT pizza_size, CAST(SUM(total_price) AS DECIMAL(10,2)) as total_revenue,  
CAST(SUM(total_price) * 100 / (SELECT SUM(total_price) from pizza_sales) AS  
DECIMAL(10,2)) AS Percentage  
FROM pizza_sales  
GROUP BY pizza_size  
ORDER BY pizza_size
```

Output

	pizza_size character varying	total_revenue numeric (10,2)	percentage numeric (10,2)
1	L	375318.70	45.89
2	M	249382.25	30.49
3	S	178076.50	21.77
4	XL	14076.00	1.72
5	XXL	1006.60	0.12

5. Total Pizzas Sold by Pizza Category

```
SELECT pizza_category, SUM(quantity) as Total_Quantity_Sold
FROM pizza_sales
GROUP BY pizza_category
ORDER BY Total_Quantity_Sold DESC
```

Output

	pizza_category text	total_quantity_sold bigint
1	Classic	14888
2	Supreme	11987
3	Veggie	11649
4	Chicken	11050

6. Top 5 Pizzas by Revenue

```
SELECT pizza_name, SUM(total_price) AS Total_Revenue
FROM pizza_sales
GROUP BY pizza_name
ORDER BY Total_Revenue DESC
LIMIT 5;
```

	pizza_name text	total_revenue double precision
1	Thai Chicken Pizza	43434.25
2	Barbecue Chicken Piz...	42768
3	California Chicken Piz...	41409.5
4	Classic Deluxe Pizza	38180.5
5	Spicy Italian Pizza	34831.25

7. Top 5 Pizzas by Quantity

```
SELECT pizza_name, SUM(Quantity) AS Total_Quantity
FROM pizza_sales
GROUP BY pizza_name
```


ORDER BY Total_Quantity **DESC**

LIMIT 5;

Output

	pizza_name text	total_quantity bigint
1	Classic Deluxe Pizza	2453
2	Barbecue Chicken Piz...	2432
3	Hawaiian Pizza	2422
4	Pepperoni Pizza	2418
5	Thai Chicken Pizza	2371

8. Top 5 Pizzas by Total Orders

SELECT pizza_name, **COUNT**(**DISTINCT** order_id) **AS** Total_Order

FROM pizza_sales

GROUP BY pizza_name

ORDER BY Total_Order **DESC**

LIMIT 5;

Output

	pizza_name text	total_order bigint
1	Classic Deluxe Pizza	2329
2	Hawaiian Pizza	2280
3	Pepperoni Pizza	2278
4	Barbecue Chicken Piz...	2273
5	Thai Chicken Pizza	2225

9. Bottom 5 Pizzas by Revenue

SELECT pizza_name, **SUM**(total_price) **AS** Total_Revenue

FROM pizza_sales

GROUP BY pizza_name

ORDER BY Total_Revenue **DESC**

LIMIT 5;

Output

	pizza_name text	total_revenue double precision
1	Brie Carre Pizza	11588.49999999999
2	Green Garden Pizza	13955.75
3	Spinach Supreme Piz...	15277.75
4	Mediterranean Pizza	15360.5
5	Spinach Pesto Pizza	15596

10. Bottom 5 Pizzas by Quantity

SELECT pizza_name, **SUM**(Quantity) **AS** Total_Quantity

FROM pizza_sales

GROUP BY pizza_name

ORDER BY Total_Quantity **ASC**

LIMIT 5;

Output

	pizza_name text	total_quantity bigint
1	Brie Carre Pizza	490
2	Mediterranean Pizza	934
3	Calabrese Pizza	937
4	Spinach Supreme Piz...	950
5	Soppressata Pizza	961

11. Bottom 5 Pizzas by Total Orders

```
SELECT pizza_name, COUNT(DISTINCT order_id) AS Total_Order
FROM pizza_sales
GROUP BY pizza_name
ORDER BY Total_Order ASC
LIMIT 5;
```

Output

	pizza_name text	total_order bigint
1	Brie Carre Pizza	480
2	Mediterranean Pizza	912
3	Calabrese Pizza	918
4	Spinach Supreme Piz...	918
5	Chicken Pesto Pizza	938

KPI measure in Power BI

1. **Total Revenue:** the sum of the total price of all pizza orders.

Total Revenue = `SUM(pizza_sales[total_price])`

2. **Average order value:** the average amount spent per order, calculated by dividing the total revenue by the total number of orders.

Average order value = `[Total Revenue]/[Total orders]`

3. **Total pizza sold:** the sum of the quantity of all pizzas sold.

Total pizza sold = `SUM(pizza_sales[quantity])`

4. **Total orders:** the total number of orders placed.

Total orders = `DISTINCTCOUNT(pizza_sales[order_id])`

5. **Average pizza per order:** the average number of pizzas sold per order, calculated by dividing the total number of pizzas sold by the total number of orders.

Average pizza per order = $\frac{[\text{Total pizza sold}]}{[\text{Total orders}]}$

Order days

Measures written to pick the first three letters on Day Name column in capital letters

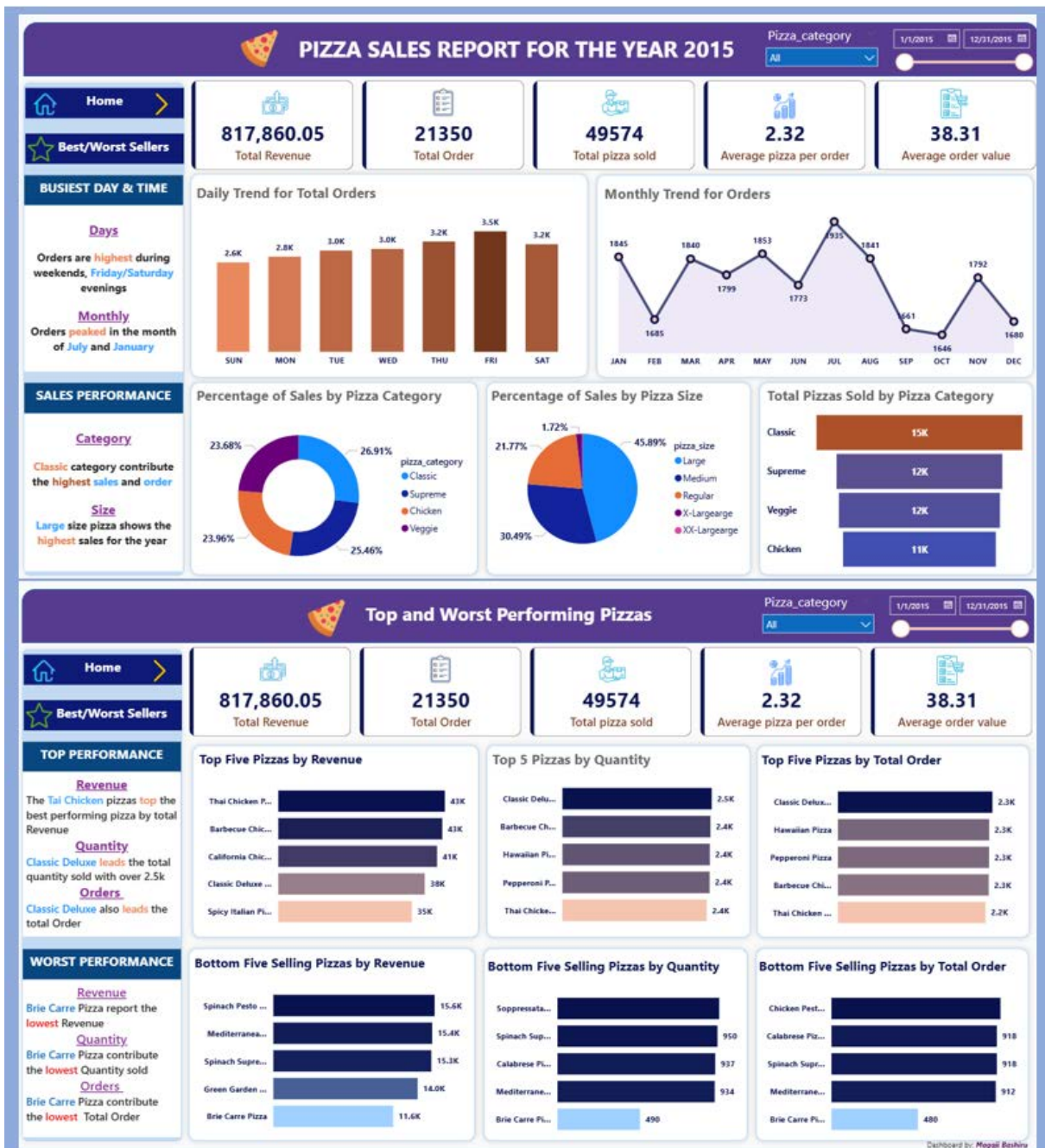
syntax

Order_days = `UPPER(LEFT(pizza_sales[Day Name], 3))`

Visualization Decisions

The dataset was also queried and visualized on Power Bi and result was compared with the results from SQL query to ensure no mismatch. The chart type used was based on the problem statement given and they include:

1. **Daily Trend for Total Orders** – a column chart is used to show the daily trend of total order over a specific time period. This chart helps to identify any patterns or fluctuations in order volume on daily basis.
2. **Monthly Trend for Total Orders** – a line chart is used here to show the monthly trend of total order throughout the year. This chart allows us identify fluctuations of orders over the year.
3. **Percentage of Sales by Pizza Category** - a doughnut-chart is used to shows the distribution of sales across different pizza category. The chart provides insights into the popularity of various pizza categories and their contribution to overall sales.
4. **Percentage of Sales by Pizza Size** - a pie-chart is used to represent the percentage of sales attributed to different pizza sizes. This chart helps us understand customer preferences for pizza size and their impact on sales.
5. **Total Pizzas Sold by Pizza Category** - a funnel chart is used to represent the total numbers of pizzas sold for each pizza category. The chart helps us to compare the sales performance of different pizza categories.
6. **Top Five Best Sellers by Revenue, Total Quantity and Total Order** - a bar-chart is used to highlight the top 5 best-selling pizzas based on the revenue, Total quantity, Total orders. This chart helps us identify the most popular pizza options.
7. **Bottom five worst sellers by Revenue, Total quantity and Total order** - a bar-chart is also used here to identify the top 5 worst-selling pizzas based on the revenue, Total quantity, Total orders. The chart enables us to identify the underperforming or less popular pizza options.



Pizza Sales Report Dashboard

Insights & Findings

- Orders are highest on Friday and lowest on Sunday
- The month of July and January have the highest order while February and October show low orders.
- Classic pizza category contributed the highest sales and order

4. Large size pizza leads total sales while X-Large and XX-Large account for the lowest sales of the year.
5. The Tai Chicken pizzas top the best performing pizza by total Revenue
6. Classic Deluxe leads the total quantity sold and total Order
7. Brie Carre Pizza contribute the lowest in Revenue, Quantity sold and Total orders

Reflection

This project reinforced the importance of patience and thorough data inspection throughout the data analysis process. Careful observation proved critical in identifying underlying issues that were not immediately apparent after initial data cleaning.

What I learned

During the course of this project, several new SQL concepts and functions were learned and applied. Notably, the CAST function was used to convert data types, particularly to present numerical results in decimal format using syntax such as *CAST(column AS DECIMAL(10,2))*. Additionally, the TO_CHAR function in PostgreSQL was explored as an equivalent to DATENAME in MS SQL, enabling the extraction of specific date components such as days of the week or months.

Further skills were developed in Power BI and Power Query, including the creation of calculated columns, measures, and the use of DAX functions to compute key performance indicators (KPIs). Conditional formatting techniques were also applied to enhance data visualization.

Challenges Encountered

A major challenge encountered during the project was related to inconsistent date formats within the dataset. Despite completing initial data cleaning, the dataset repeatedly failed to load into the PostgreSQL database. After extensive troubleshooting, it was discovered that the date column contained inconsistent formats. Once standardized, the data was successfully imported.

Similar issue arose during data visualization in Power BI, where incorrect date formats led to errors and inconsistencies. This also affected the alignment between Power BI results and SQL query outputs, which initially did not match. These discrepancies were ultimately traced back to the same date formatting issue and were resolved after proper standardization.

Another challenge faced was the persistent sign-in pop-up on the Power BI interface, which disrupted workflow and caused delays. Despite multiple attempts, a solution to this issue has not yet been identified.

Future Improvements

One of the most important insights gained from this project is the critical role of date formatting in data analysis. Improper handling of date fields can lead to failed data imports,

incorrect analysis, and misinterpretation of results. This highlights the need for extra caution when working with date-related data.

If repeating this project, greater emphasis will be placed on thoroughly validating data especially date fields before proceeding with analysis or database integration. Early detection of such issues would significantly reduce troubleshooting time and improve overall workflow efficiency.

References

Data source: <https://www.kaggle.com/datasets/nextmillionaire/pizza-sales-dataset>

Replicated Concept: <https://www.youtube.com/watch?v=V-s8c6jMRN0&list=PPSV>